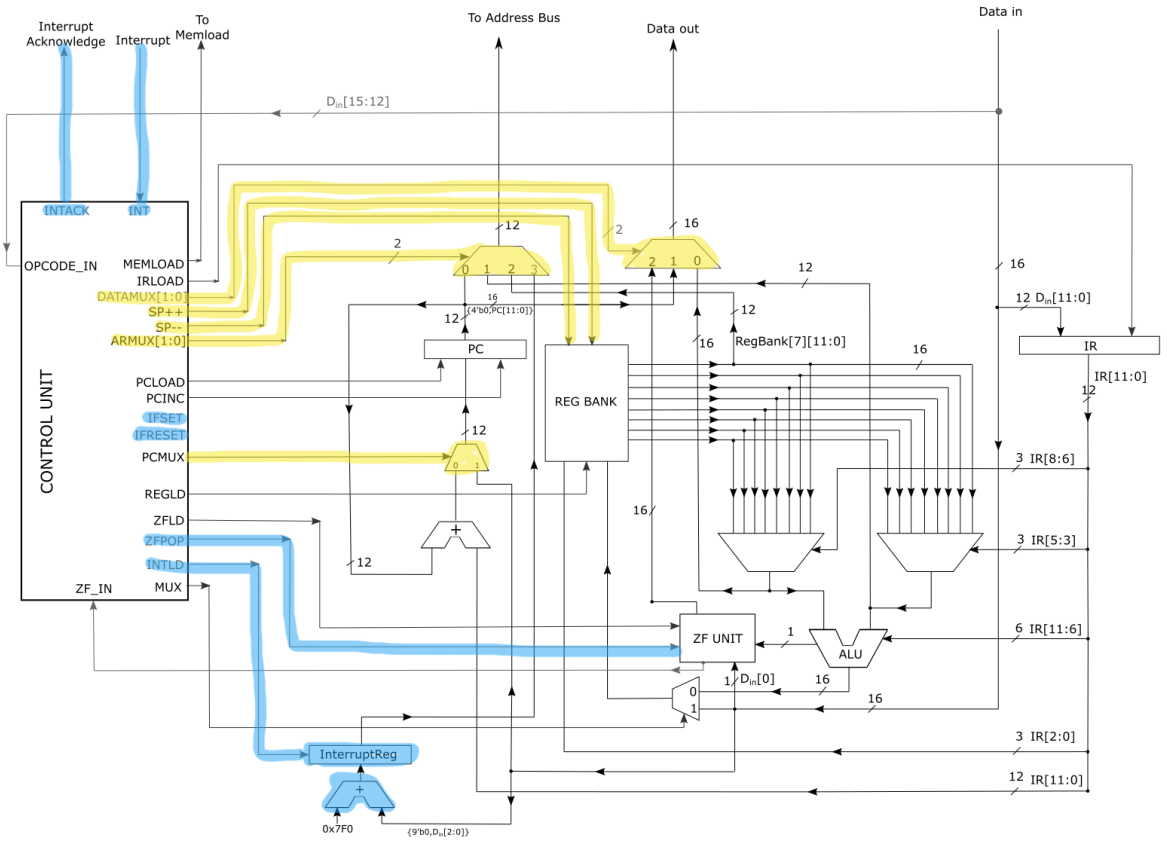
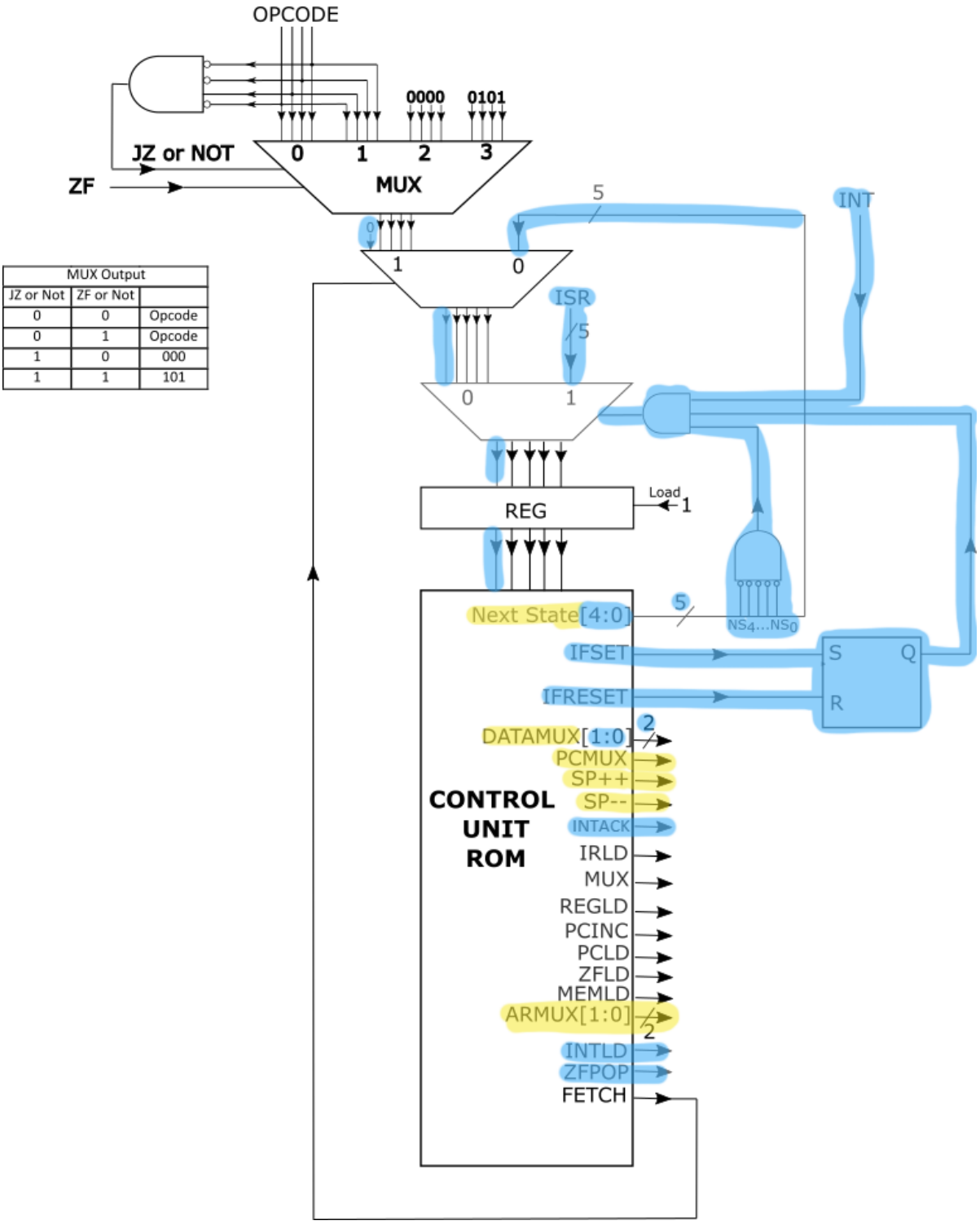
Microprocessors

Mammal and Bird Hardware Comparison

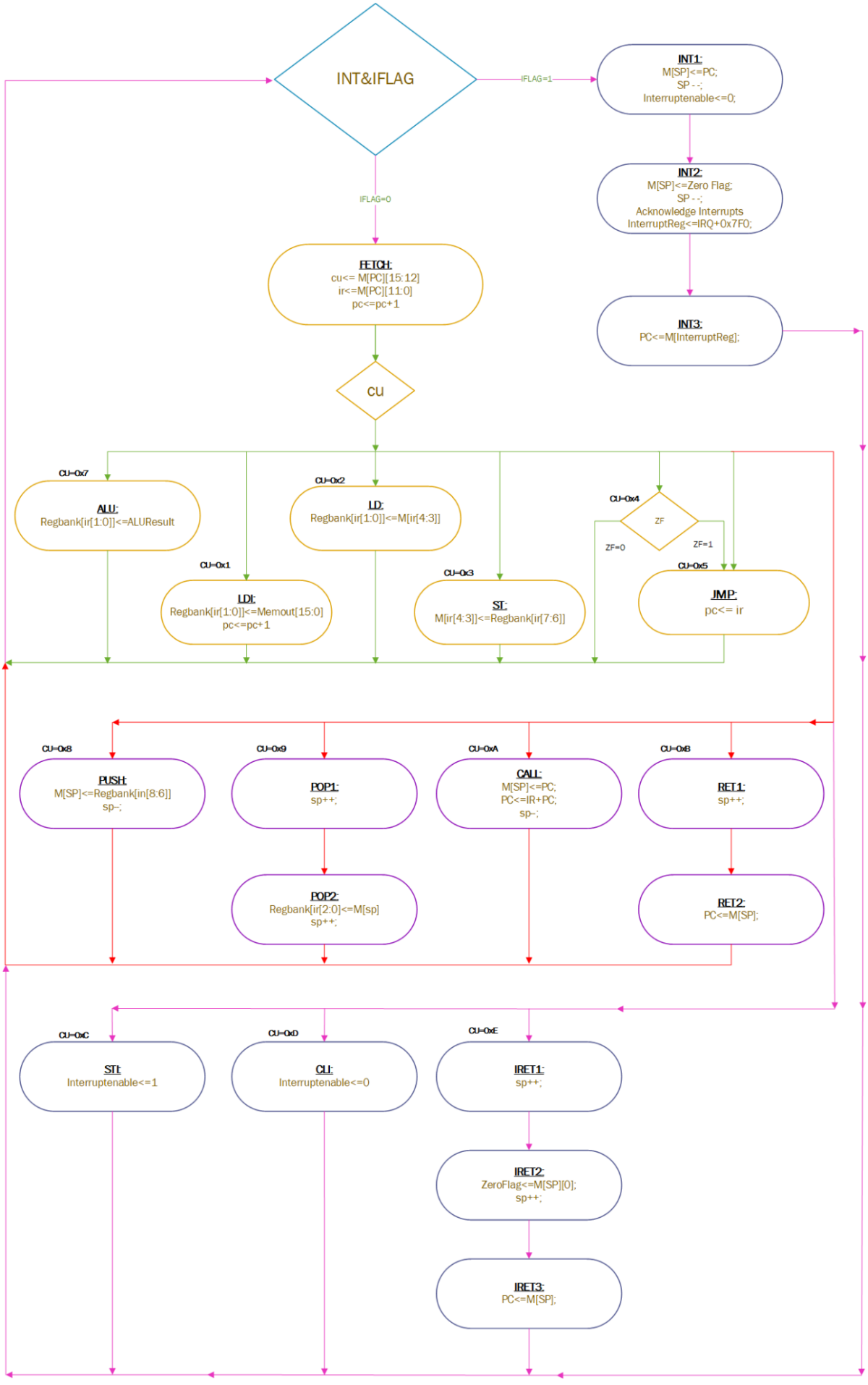




MUX Output		
JZ or Not	ZF or Not	
0	0	Opcode
0	1	Opcode
1	0	000
1	1	101

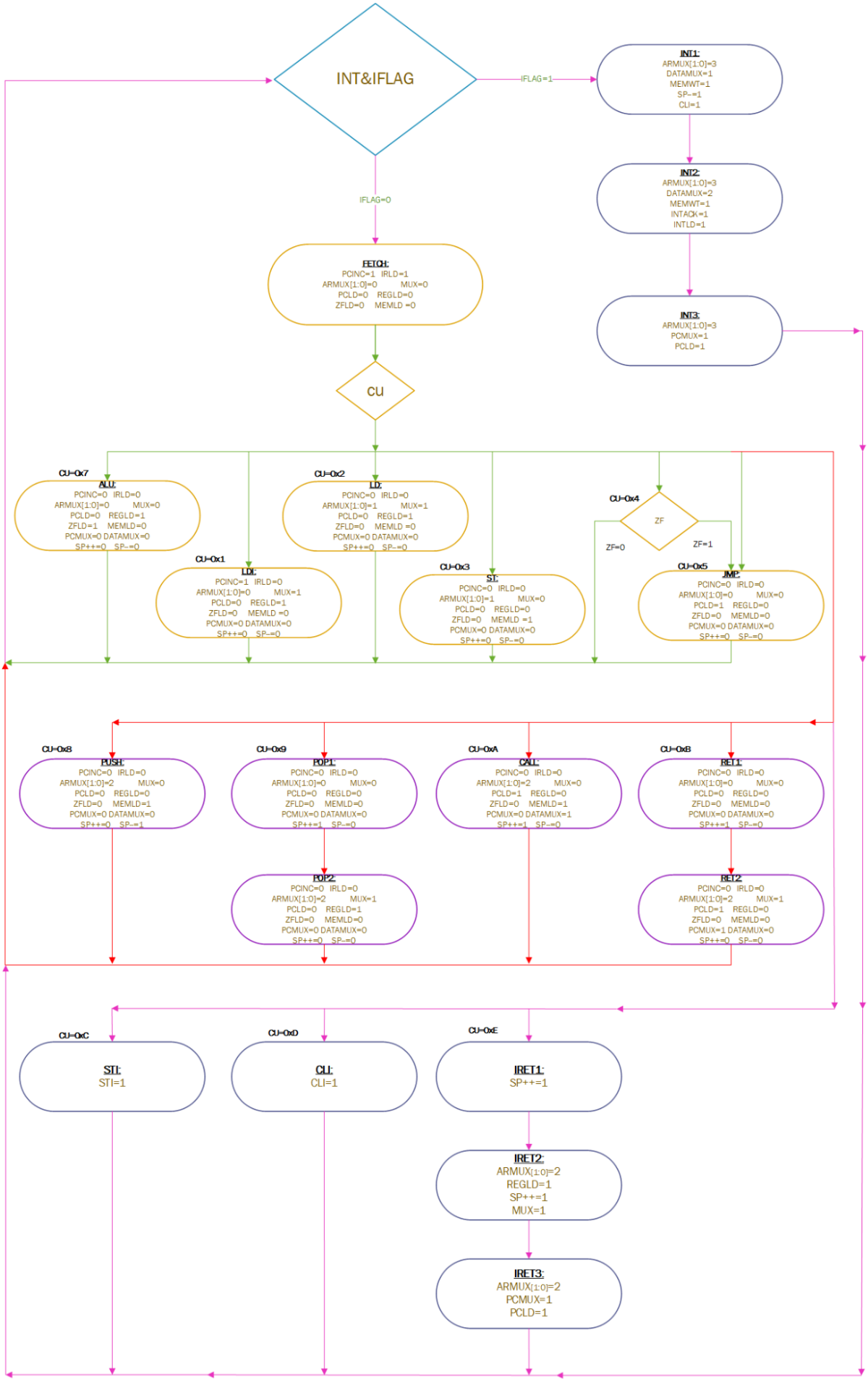
Mammal

Finite State Machine


INT&IFLAG
IFLAG=1
INT1:
M[SP]<=PC;
SP - -;
Interruptenable<=0;
INT2:
M[SP]<=Zero Flag;
SP - -;
Acknowledge Interrupts
InterruptReg<=IRQ+0x7F0;
INT3:
PC<=M[InterruptReg];
IFLAG=0
FETCH:
cu<= M[PC][15:12]
ir<=M[PC][11:0]
pc<=pc+1
cu
CU=0x7
ALU:
Regbank[ir[1:0]]<=ALUResult
CU=0x2
LD:
Regbank[ir[1:0]]<=M[ir[4:3]]
CU=0x4
ZF
ZF=0
ZF=1
CU=0x1
LDI:
Regbank[ir[1:0]]<=Memout[15:0]
pc<=pc+1
CU=0x3
ST:
M[ir[4:3]]<=Regbank[ir[7:6]]
CU=0x5
JMP:
pc<= ir
CU=0x8
PUSH:
M[SP]<=Regbank[in[8:6]]
sp--;
CU=0x9
POP1:
sp++;
POP2:
Regbank[ir[2:0]]<=M[sp]
sp++;
CU=0xA
CALL:
M[SP]<=PC;
PC<=IR+PC;
sp--;
CU=0xB
RET1:
sp++;
RET2:
PC<=M[SP];
CU=0xC
STI:
Interruptenable<=1
CU=0xD
CLI:
Interruptenable<=0
CU=0xE
IRET1:
sp++;
IRET2:
ZeroFlag<=M[SP][0];
sp++;
IRET3:
PC<=M[SP];

Control Signals

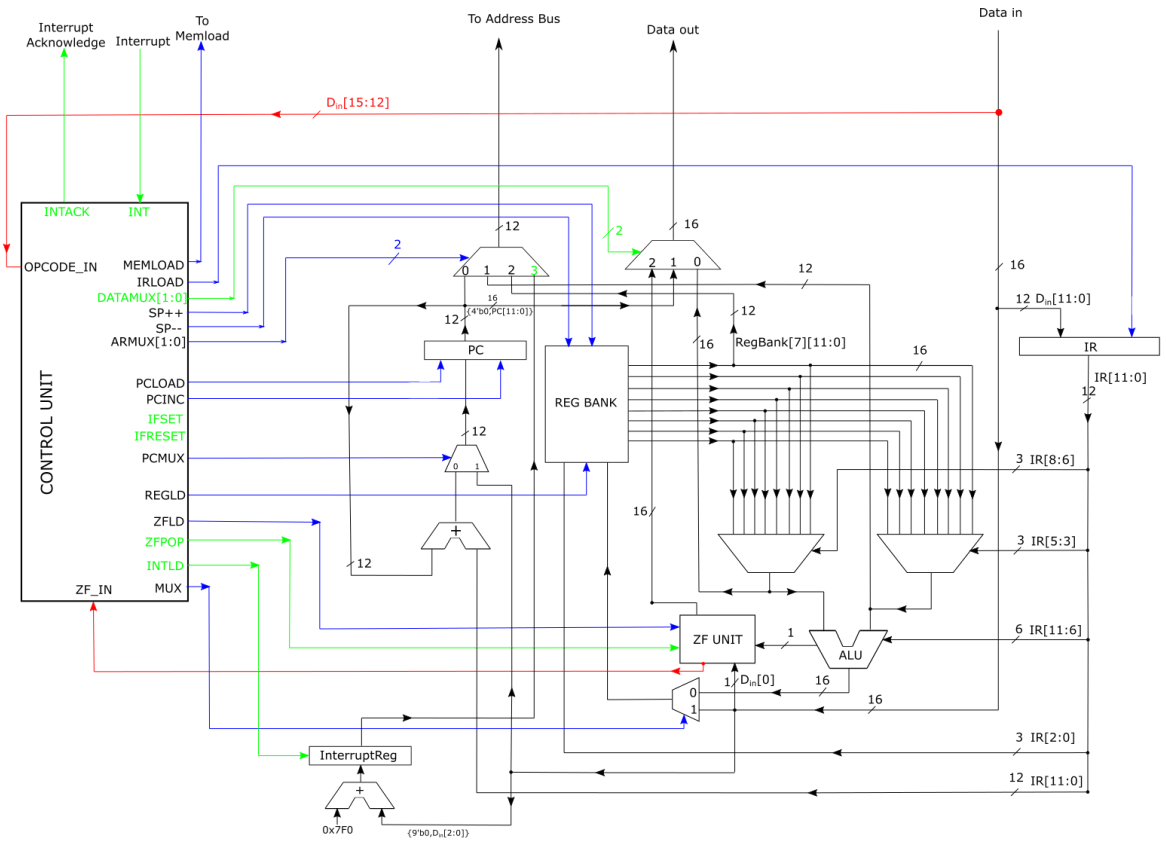
State	Signals
FETCH	FETCH, PCINC, IRLD
LDI	PCINC, REGLD, MUX
LD	ARMUX, MUX
ST	ARMUX, MEMLD
JMP	PCLD
JMPZ	
ALU	REGLD, ZFLD
PUSH	ARMUX=2, MEMLD, SP–
POP1	SP++
CALL	ARMUX=2, PCLD, MEMLD, SP++
RET1	SP++
STI	IFSET
CLI	IFRESET
IRET1	SP++
IRET2	ARMUX=2, REGLD, SP++, MUX
IRET3	ARMUX=2, PCMUX, PCLD
POP2	ARMUX=2, MUX
RET2	ARMUX=2, MUX, PCMUX
INT1	ARMUX=3, DATAMUX, MEMLD, SP–, IFRESET
INT2	ARMUX=3, DATAMUX=2, MEMLD, SP–, INTACK, INTLD, ZFPOP
INT3	ARMUX=3, PCMUX, PCLD



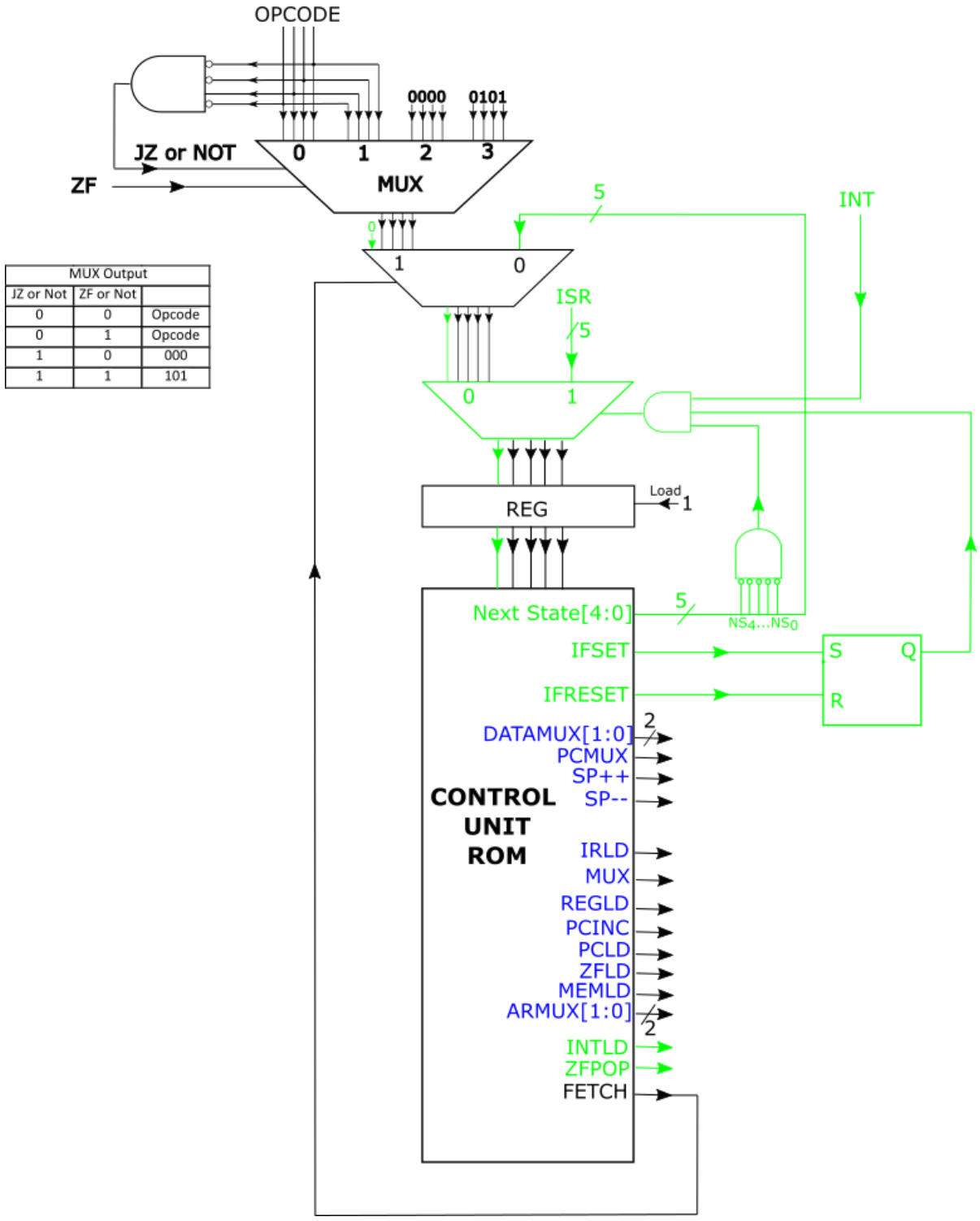
In each FSM state, unmentioned signals are assumed to be zero.

STI=IFSET, CLI=IFRESET

Hardware with Control Unit

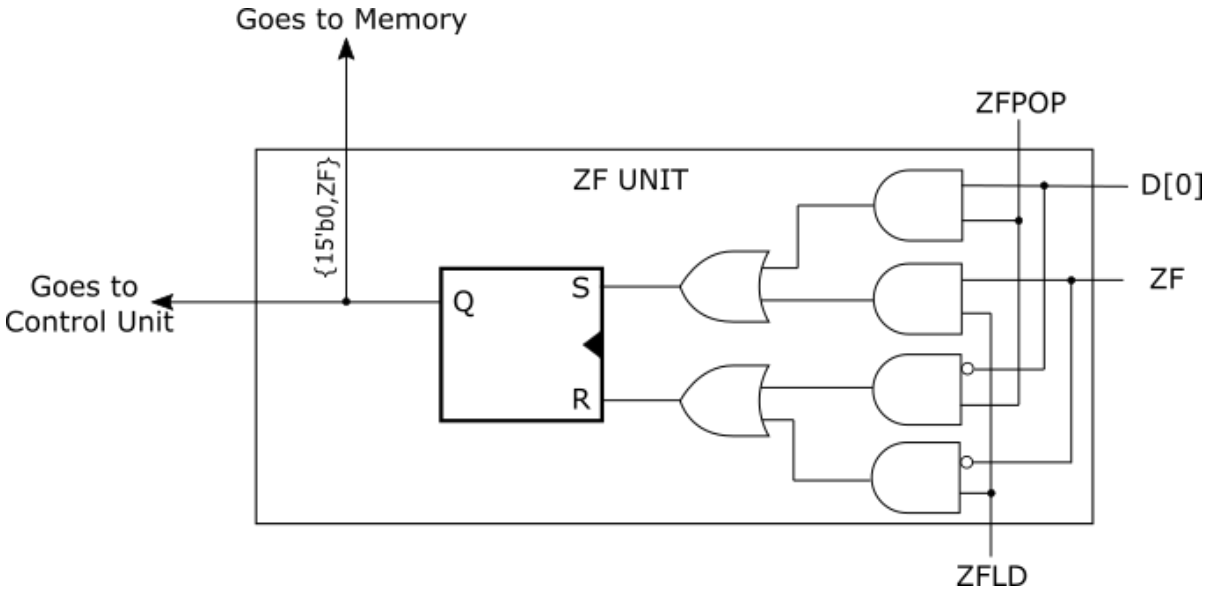


Control Unit

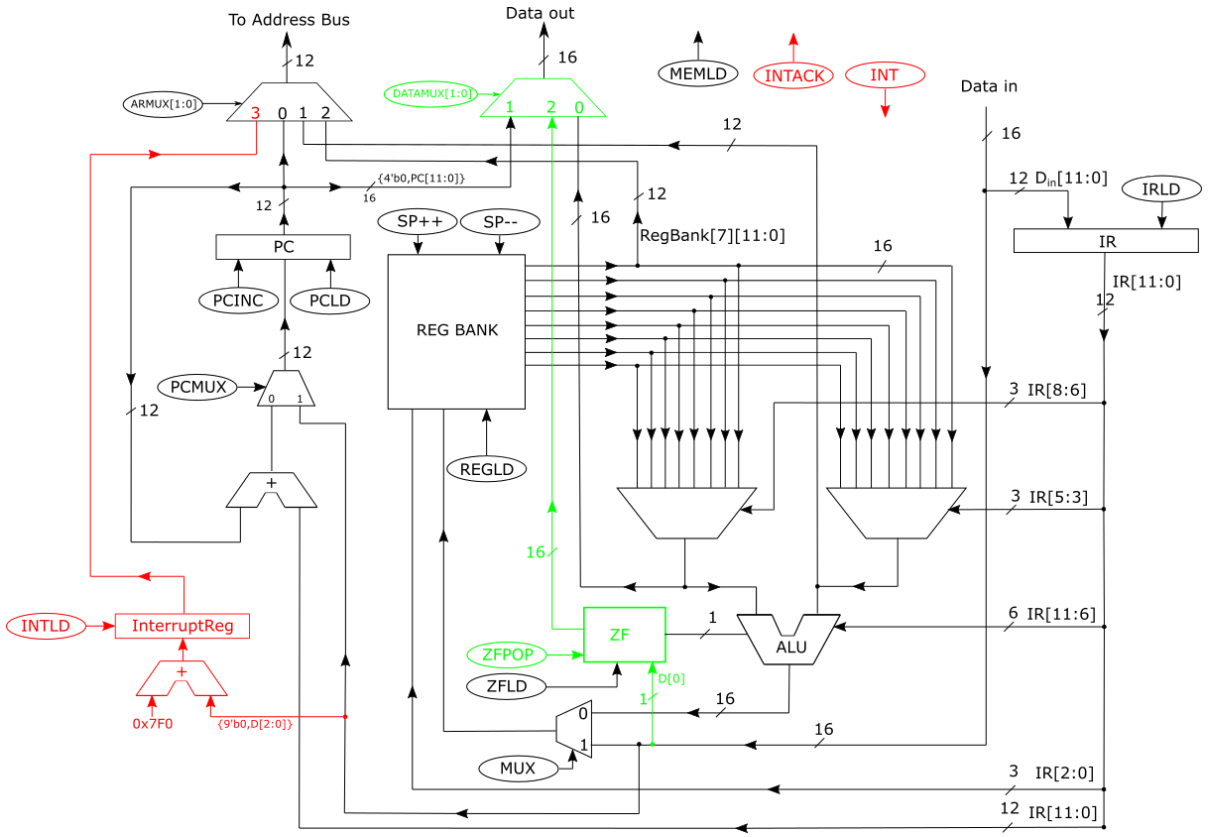


MUX Output		
JZ or Not	ZF or Not	
0	0	Opcode
0	1	Opcode
1	0	000
1	1	101

ZF Unit



Hardware Design



Microcode

Adress	Inst Name	D24	D23	D22	D21	D20	D19	D18	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D[4:0]	
		INTACK	FETCH	ZFPOP	INTLD	ARMUX[1]	ARMUX[0]	MEMLD	ZFLD	PCLD	PCINC	REGLD	MUX	IRLD	SP--	SP++	PCMux	Datamux[1]	Datamux[0]	IfReset	IfSet	Next State[4:0]	HEX Conversion
0X00	FETCH		1								1			1								0X00	0x0809000
0X01	LDI										1	1	1									0X00	0x000E000
0X02	LD						1					1	1									0X00	0x0086000
0X03	ST						1	1														0X00	0x00C0000
0X04																						0X00	0x0000000
0X05	JMP									1												0X00	0x0010000
0X06																						0X00	0x0000000
0X07	ALU								1			1										0X00	0x0024000
0X08	PUSH					1		1							1							0X00	0x0140800
0X09	POP1															1						0x0F	0x000040F
0X0A	CALL					1		1		1					1				1			0X00	0x0150880
0X0B	RET1															1						0x10	0x0000410
0X0C	STI																				1	0X00	0x0000020
0X0D	CLI																			1		0X00	0x0000040
0X0E	IRET1															1						0x11	0x0000411
0X0F	POP2					1						1	1									0X00	0x0106000
0X10	RET2					1				1			1				1					0X00	0x0112200
0X11	IRET2			1		1							1			1						0x12	0x0502412
0X12	IRET3					1				1							1					0X00	0x0110200
0X13	INT1					1		1							1				1	1		0x14	0x01408D4
0X14	INT2	1			1	1		1							1			1				0x15	0x1340915
0X15	INT3					1	1			1							1					0X00	0x0190200

Address	OPCODE	INS	FETCH	DATAMUX(1)	DATAMUX(0)	PCMUX	SP++	SP–	PCLD	PCINC	ARMUX(1)	ARMUX(0)	REGLD	ZFLD	MUX	MEMLD	IRLD	INTLD	ZFPOP	IFSET	IFRESET	INTACK	Next State	Hex Code
0	00000	FETCH	1	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	11111	102041F
1	00001	LDI	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	0	0	0	0	0	00000	0025000
2	00010	LD	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	00000	000D000
3	00011	ST	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	00000	0008800
4																							00000	0000000
5	00101	JMP	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	00000	0040000
6		JMPZ																					00000	0000000
7	00111	ALU	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	00000	0006000
8	01000	PUSH	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	00000	0090000
9	01001	POP1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	10001	0100011
A	01010	CALL	0	0	1	0	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	00000	6550800
B	01011	RET1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	10010	0100012
C (12)	01100	STI	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	00000	0000080
D	01101	CLI	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	00000	0000040
E (14)	01110	IRET1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	01111	010000F
F	01111	IRET2	0	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	10000	0111110
16	10000	IRET3	0	0	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	00000	0250000
17	10001	POP2	0	0	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	00000	0015000
18	10010	RET2	0	0	0	1	0	0	1	0	1	0	0	0	1	0	0	0	0	0	0	0	00000	0251000
19	10011	INT1	0	0	1	0	0	1	0	0	1	0	0	0	0	1	0	0	0	0	1	0	10100	0490854
20	10100	INT2	0	1	0	0	0	1	0	0	1	0	0	0	0	1	0	1	0	0	0	1	10101	0890A35
21	10101	INT3	0	0	0	1	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	00000	0258000

	fetch	datamux1	datamux0	pcmux	sp++	sp--	pcld	pcinc	armux1	armux0	regld	zfld	mux	memld	irld	zfpop	sti	cli	intack	intld	ns4	ns3	ns2	ns1	ns0	
0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	1	1	1	1	1	fetch
1	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	ldi
2	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	ld
3	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	st
4																					0	0	0	0	0	
5	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	jmp
6																					0	0	0	0	0	
7	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	alu
8	0	0	0	0	0	1	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	push
9	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	pop1
A	0	0	1	0	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	call
B	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	ret1
C	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	sti
D	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	cli
E	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	iret1
F	0	0	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	pop2
10	0	0	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	ret2
11	0	0	0	0	1	0	0	0	1	0	0	1	1	0	0	1	0	0	0	0	1	0	0	1	0	iret2
12	0	0	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	iret3
13	0	0	1	0	0	1	0	0	1	0	0	0	0	1	0	0	0	1	0	0	1	0	1	0	0	int1
14	0	1	0	0	0	1	0	0	1	0	0	1	0	1	0	0	0	0	1	1	1	0	1	0	1	int2
15	0	0	0	1	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	int3

Instruction Set

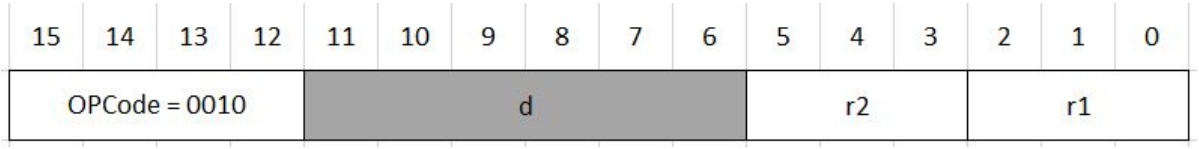
LD

Load instruction transfers

- the contents of a memory location (which is a 16-bit number)
- into a register of Reptile.

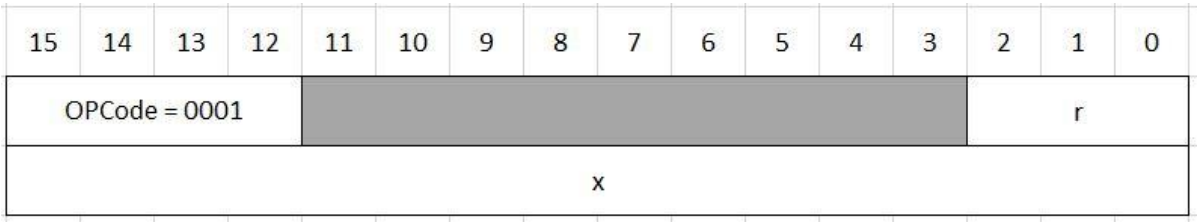
The assembly format of the Load instruction is as follows:

LD r1 r2



LDI

LDI r,x



ALU Instructions

Instruction Format

ASSEMBLY	EXPLANATION	C EQUIVALENT
ADD r1 r2 r3	Adds registers r2 and r3, puts the result into register r1	r1=r2+r3
SUB r1 r2 r3	Subtracts register r3 from register r2, puts the result into register r1	r1=r2-r3
AND r1 r2 r3	Ands registers r2 and r3, puts the result into register r1	r1=r2&r3
OR r1 r2 r3	Or's registers r2 and r3, puts the result into register r1	r1=r2 | r3
XOR r1 r2 r3	Xor's registers r2 and r3, puts the result into register r1	r1=r2^r3
NOT r1 r2	Negates the contents of registers r2, puts the result into register r1	r1=!r2
MOV r1 r2	Transfers the contents of registers r2 into register r1	r1=r2
INC r1	Increments the contents of registers r1	r1++
DEC r1	Decrements the contents of registers r1	r1–

ALU Codes

Instruction	ALU CODE	ALU CODE (Circuit)	ALU CODE
ADD	0000	**000**000	0
SUB	0001	**001**000	16

AND	0010	**010**000	32
OR	0011	**011**000	48
XOR	0100	**100**000	64
NOT	0101	00**0101**	3
MOV	0110	00**0110**	6
INC	0111	00**0111**	7
DEC	1000	00**1000**	8

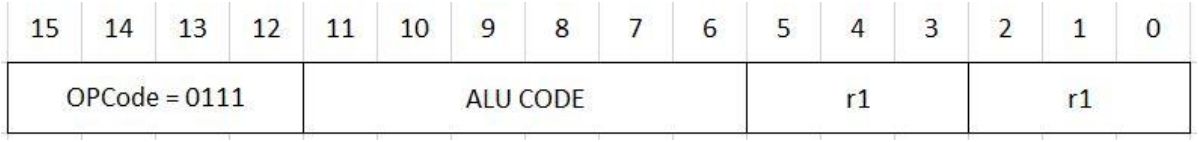
INC

INC r1



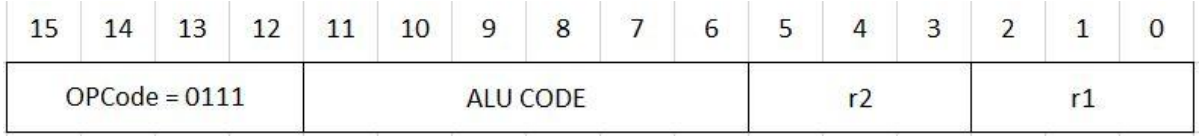
DEC

DEC r1



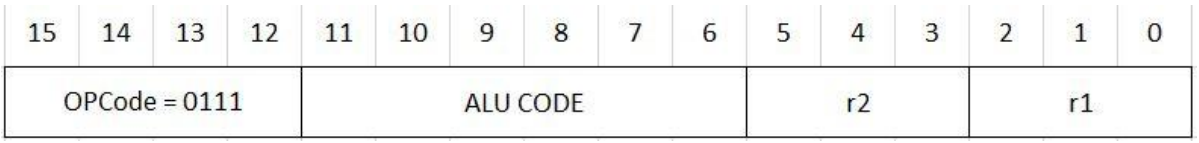
MOV

MOV r1 r2

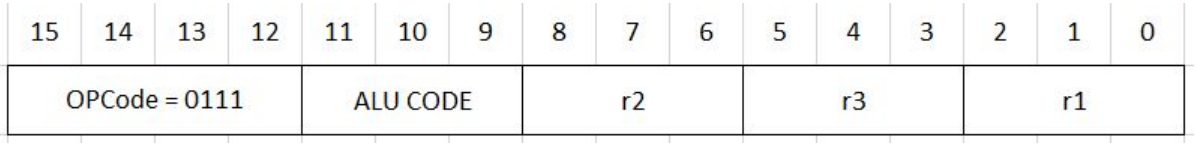


NOT

NOT r1 r2

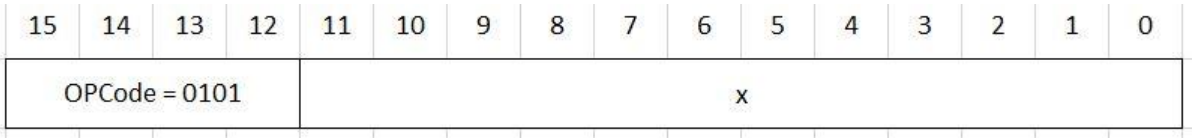


ADD, SUB, AND, OR, XOR



JUMP

JMP instruction is unconditional: it always jumps to the given target location within the code



JZ

JZ instruction is conditional: it jumps to the given target location within the code only if the result of the last ALU instruction is zero. Otherwise, the next instruction in the program is executed.



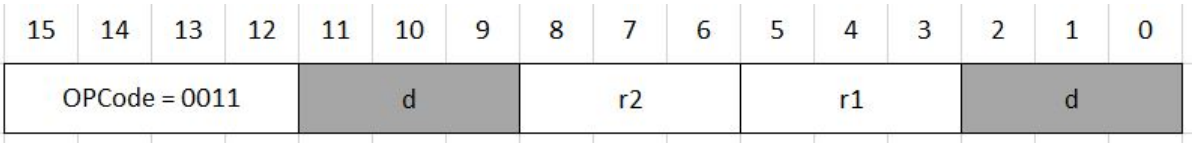
ST

Store instruction stores

- The contents of a register
- Into a memory location, which is pointed by another register.

The assembly format of the Store instruction is as follows:

ST r1 r2



Here r1 and r2 are two-bit fields denoting two registers:

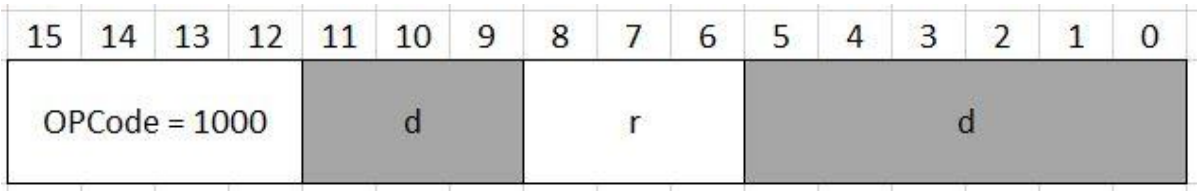
- r1 is a register which “points” to memory: It contains the address of the memory location to which the data is to be loaded .
- r2 is the register which contains the data to be stored in memory.

PUSH

Assembly format: Push r

Writes the contents of the register r into the memory location pointed by the SP. Then decrements SP by one.

Machine code format:

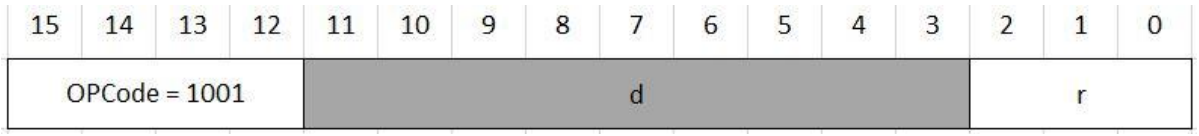


POP

Assembly format: Pop r

Pop increments SP by 1, then transfers the content of the memory location pointed by SP into register r.

Machine code format:



CALL

Assembly format: Call x

Pushes the content of the Program Counter (PC) into the stack and jumps to memory address PC+x, i.e, loads the number PC+x into PC. (PC<=PC+x). Recall that PC is already incremented when we start to execute the Call instruction.

Machine code format:

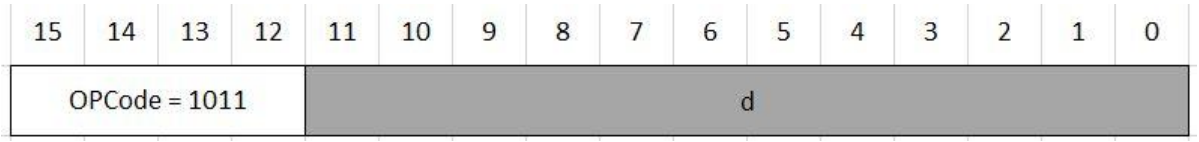


RET

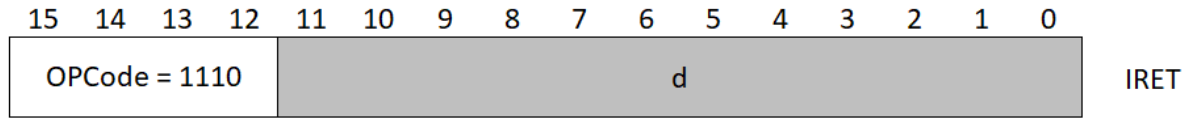
Assemby format: Ret

Pops the top of the stack into PC.

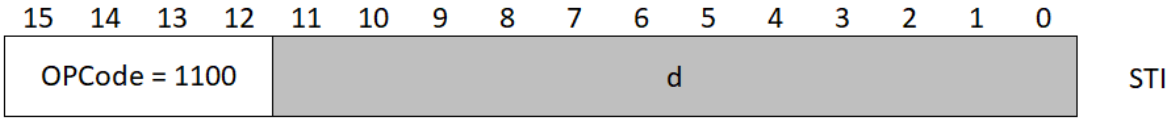
Machine code format:



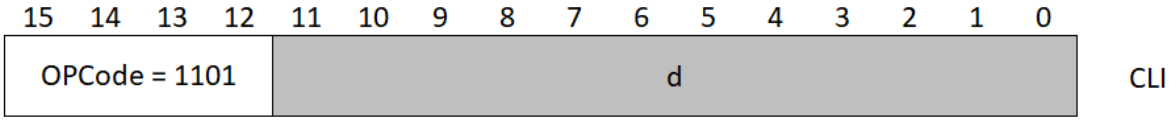
IRET



STI

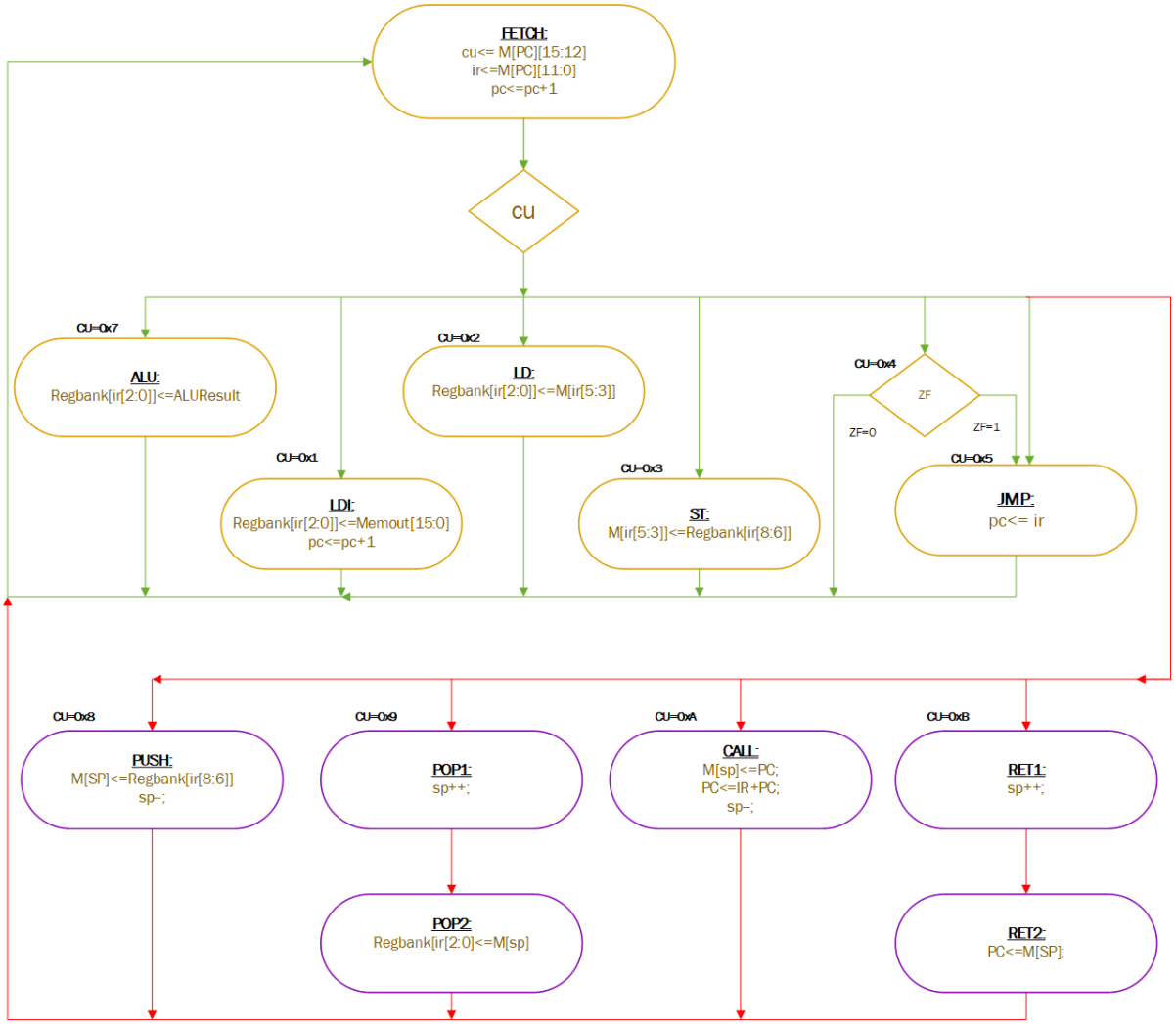

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
OPCode = 1100
d
STI

CLI

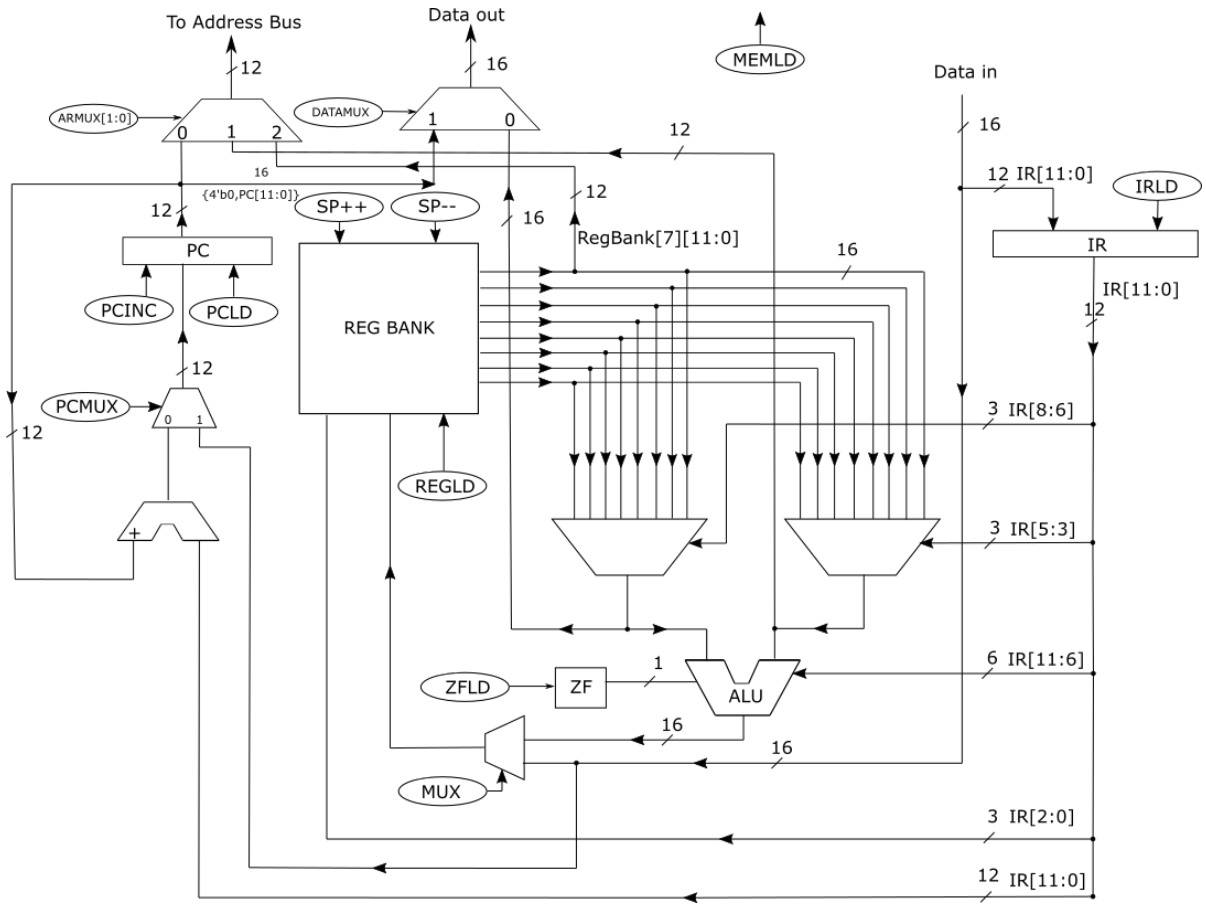

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
OPCode = 1101
d
CLI

Bird

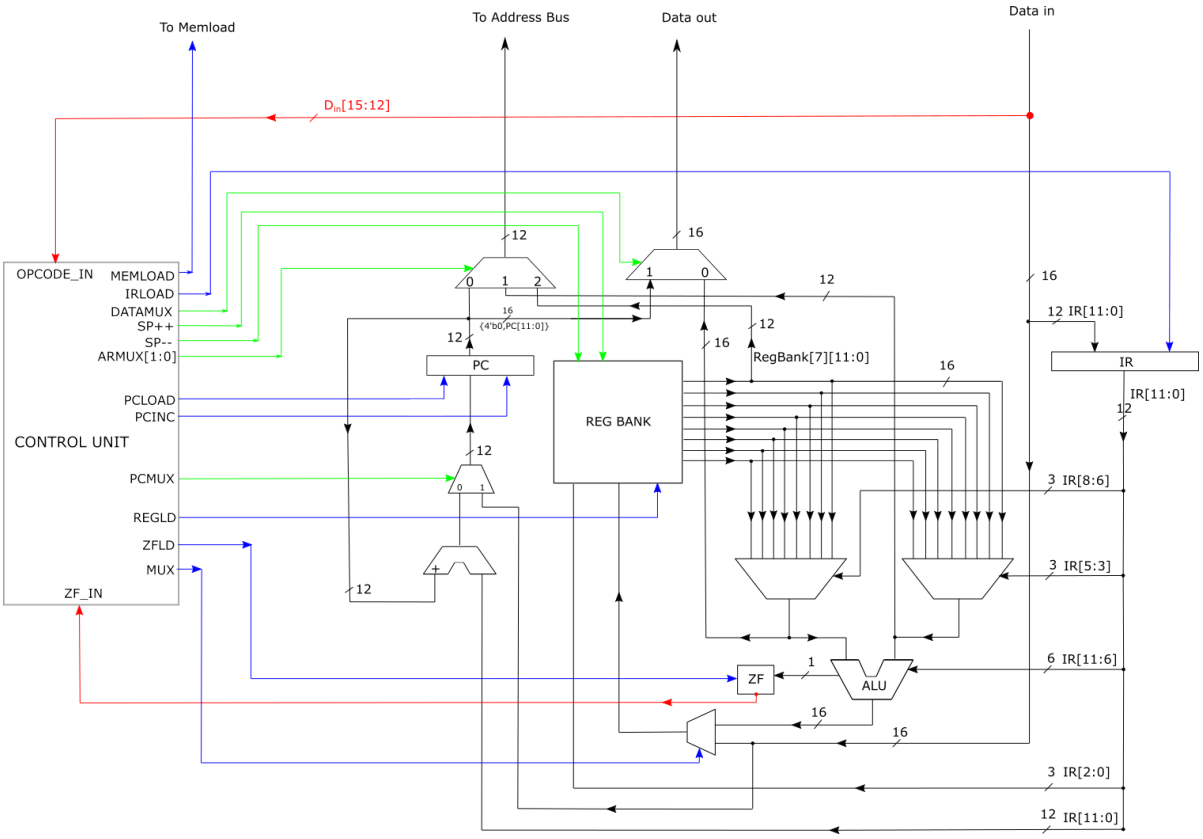
Finite State Machine



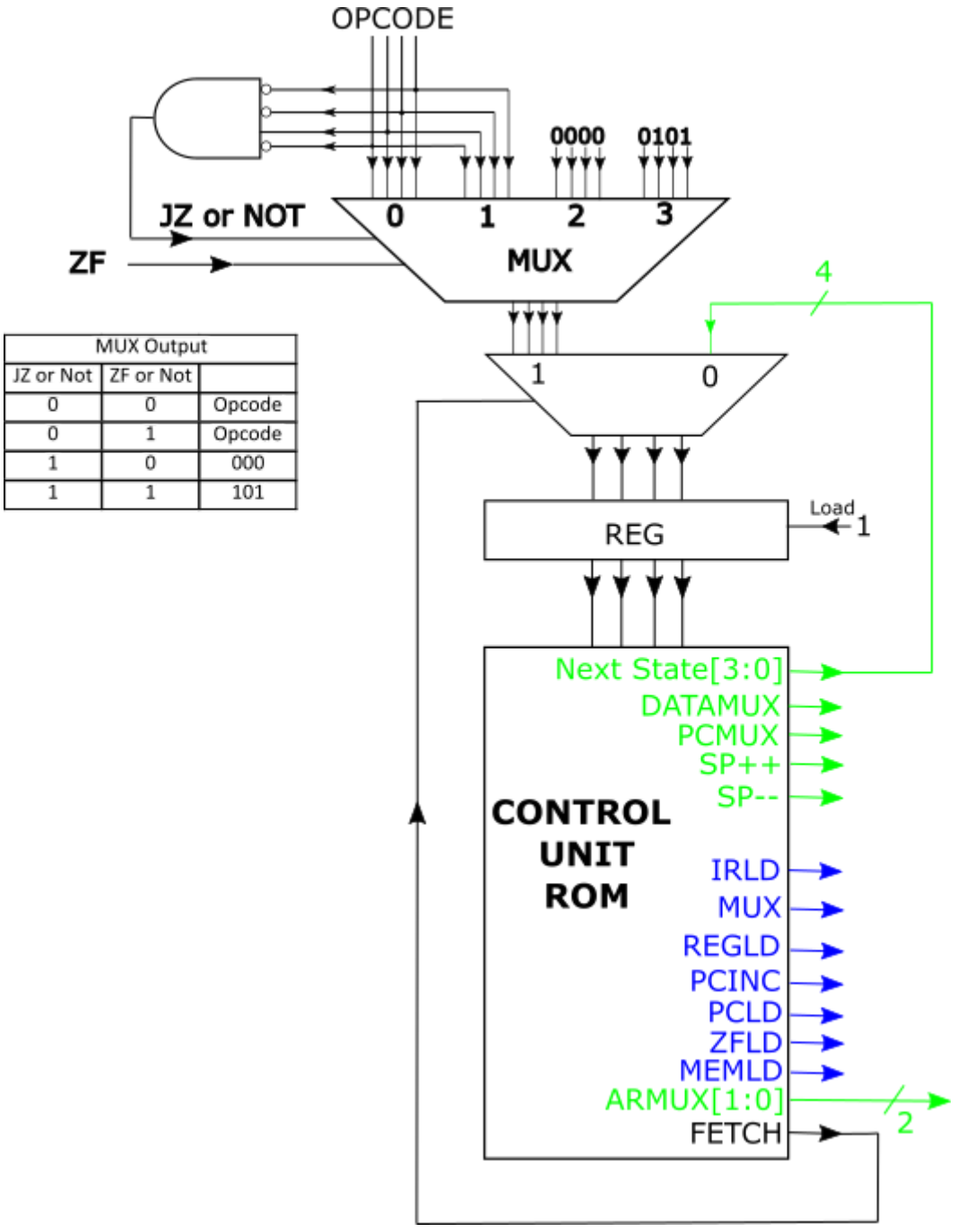
Hardware Design



Control Unit Connection

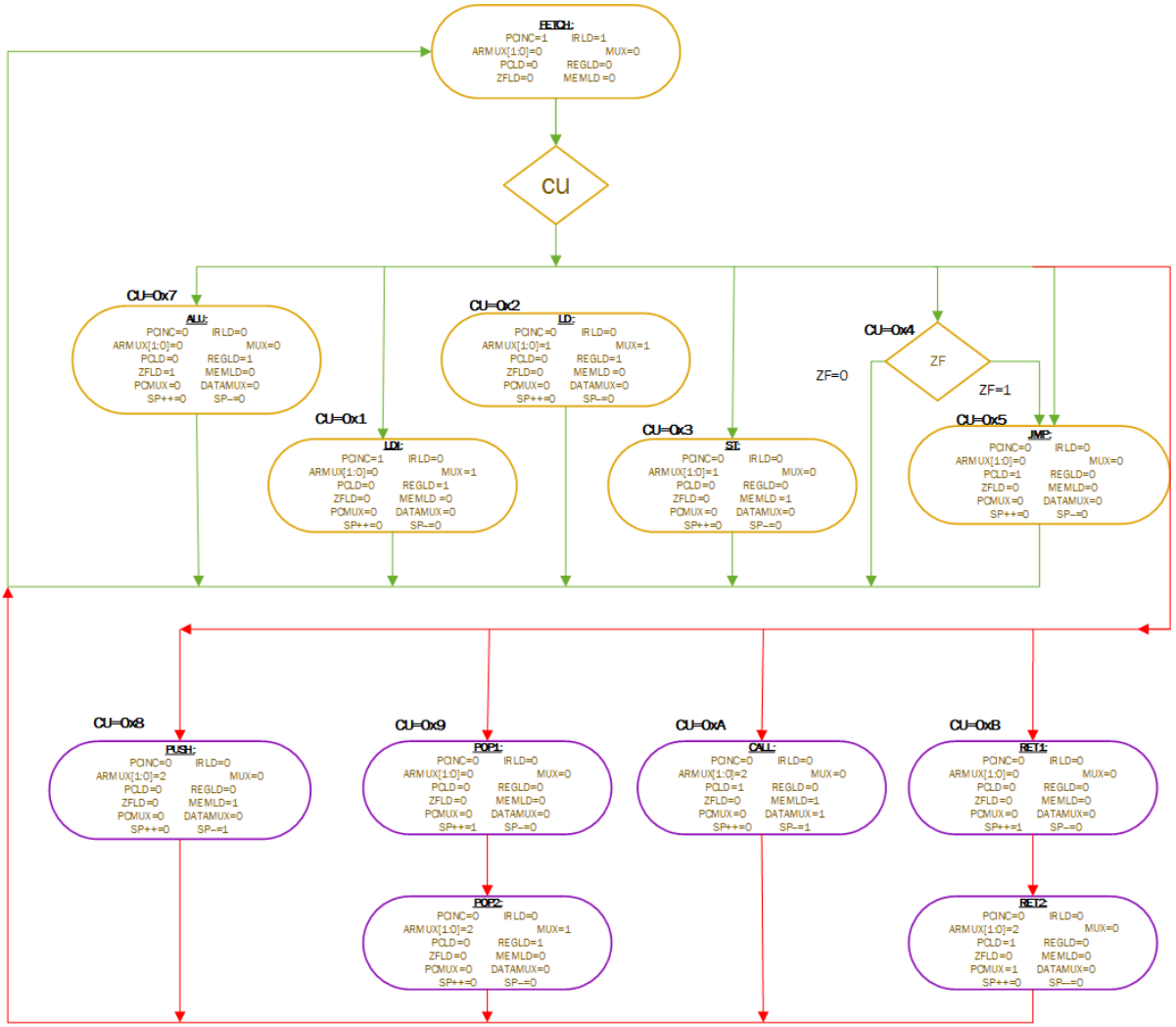


Control Unit Interior



MUX Output		
JZ or Not	ZF or Not	
0	0	Opcode
0	1	Opcode
1	0	000
1	1	101

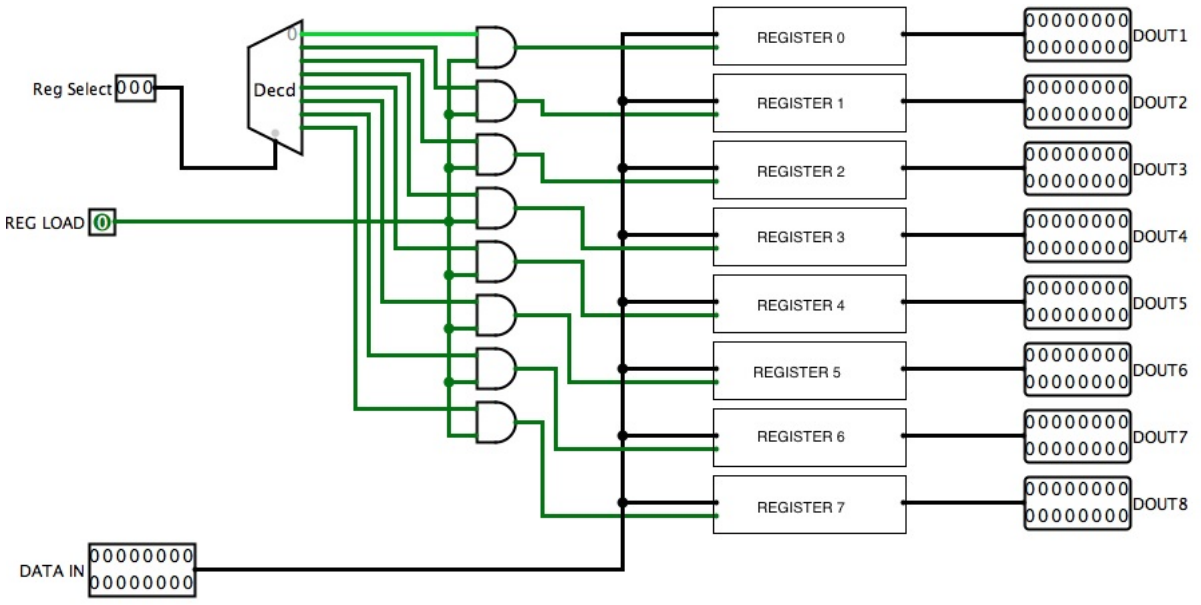
Control Signals



Microcode

Address	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D[3:0]	
	FETCH	DATAMUX	PCMUX	SP++	SP--	PCLD	PCINC	ARMUX[1]	ARMUX[0]	REGLD	ZFLD	MUX	MEMLD	IRLD	Next State[3:0]	HEX CONVERSION
0	1						1							1	0X0	0x2810
1							1			1		1			0X0	0x940
2									1	1		1			0X0	0x340
3									1				1		0X0	0x220
4															0X0	0x0
5						1									0X0	0x1000
6															0X0	0x0
7										1	1				0X0	0x180
8					1			1					1		0X0	0x2420
9				1											0xC	0x400C
A		1			1	1		1					1		0x0	0x13420
B				1											0xD	0x400D
C								1		1		1			0x0	0x540
D			1			1		1							0X0	0x9440

Register Bank



Verilog

```
module main_module ( input clk, input ps2c, input ps2d, output logic [3:0] grounds, output
logic [6:0] display, input pushbutton //may be used as clock );
logic [15:0] data_all;
logic [3:0] keyout;
logic ack;
//memory map is defined here
localparam BEGINMEM=12'h000,
ENDMEM=12'h1ff,
KEYBOARD=12'h900,
SEVENSEG=12'hb00;
// memory chip
logic [15:0] memory [0:127];
// cpu's input-output pins
logic [15:0] data_out;
logic [15:0] data_in;
logic [11:0] address;
logic memwt;
sevensegment ss1 (//to be added);
keyboard kb1(//to be added);
bird br1 (//to be added);
//multiplexer for cpu input
always_comb
if ( (BEGINMEM<=address) && (address<=ENDMEM) )
begin
data_in=memory[address];
end
else if (address==KEYBOARD+1)
begin
//to be added
end
else if (address==KEYBOARD)
begin
//to be added
end
else begin
data_in=16'hf345; //any number
end
//multiplexer for cpu output
always_ff @(posedge clk) //data output port of the cpu
if (memwt)
if ( (BEGINMEM<=address) && (address<=ENDMEM) )
memory[address]<=data_out;
else if ( SEVENSEG==address)
data_all<=data_out;
initial
begin
$readmemh("ram.dat", memory);
end
endmodule
```

Reptile

Verilog

CPU

```
//Reptile design with 8 Registers.
// This module also sends register #0 to main module as an output
module reptile (
                    input clk,
                    input [15:0] data_in,
                    output [15:0] data_out,
                    output logic [11:0] address,
                    output memwt,
                    output [15:0] reg0
                    );
   logic [11:0] pc, ir; //program counter, instruction register
   logic [4:0]  state; //FSM
   logic [15:0] regbank [7:0];//registers
   logic zeroflag; //zero flag register
   logic [15:0] result; //output for result
    localparam   FETCH=4'b0000,
                 LDI=4'b0001,
                 LD=4'b0010,
                 ST=4'b0011,
                 JZ=4'b0100,
                 JMP=4'b0101,
                 ALU=4'b0111;

   logic zeroresult; //zeroflag value
   always_ff @(posedge clk)
       case(state)
           FETCH:
           begin
               if ( data_in[15:12]==JZ) // if instruction is jz
                   if (zeroflag)  //and if last bit of 7th register is 0 then jump
to jump instruction state
                       state <= JMP;
                   else
                       state <= FETCH; //stay here to catch next instruction
               else
                   state <= data_in[15:12]; //read instruction opcode and jump the
state of the instruction to be read
               ir<=data_in[11:0]; //read instruction details into instruction
register
               pc<=pc+1; //increment program counter
           end
           LDI:
           begin
```

```
                regbank[ ir[2:0] ] <= data_in; //if inst is LDI get the destination
register number from ir and move the data in it.
                pc<=pc+1; //for next instruction (32 bit instruction)
                state <= FETCH;
            end
             LD:
                begin
                    regbank[ir[2:0]] <= data_in;
                    state <= FETCH;
                end
            ST:
                begin
                          state <= FETCH;
                end
            JMP:
                begin
                    pc <= pc+ir;
                    state <= FETCH;
                end
            ALU:
                begin
                    regbank[ir[2:0]]<=result;
                    zeroflag<=zeroresult;
                    state <= FETCH;
                end
        endcase
    always_comb
        case (state)
            LD:      address=regbank[ir[5:3]][11:0];
            ST:      address=regbank[ir[5:3]][11:0];
            default: address=pc;
         endcase
assign memwt=(state==ST);
assign data_out = regbank[ir[8:6]];
    always_comb
        case (ir[11:9])
            3'h0: result = regbank[ir[8:6]]+regbank[ir[5:3]]; //000
            3'h1: result = regbank[ir[8:6]]-regbank[ir[5:3]]; //001
            3'h2: result = regbank[ir[8:6]]&regbank[ir[5:3]]; //010
            3'h3: result = regbank[ir[8:6]]|regbank[ir[5:3]]; //011
            3'h4: result = regbank[ir[8:6]]^regbank[ir[5:3]]; //100
            3'h7: case (ir[8:6])
                        3'h0: result = !regbank[ir[5:3]];
                        3'h1: result = regbank[ir[5:3]];
                        3'h2: result = regbank[ir[5:3]]+1;
                        3'h3: result = regbank[ir[5:3]]-1;
                        default: result=16'h0000;
                  endcase
            default: result=16'h0000;
        endcase
```

```
  assign zeroresult = ~|result;
   assign reg0=regbank[0];
   initial begin;
            state=FETCH;
                zeroflag=0;
                pc=0;
            end
endmodule
```